Практическое задание

Для выполнения вам понадобится локальная база PostgreSQL (можно в Docker).

Задача 1: Оконные функции

Создайте таблицу transactions (id, user_id, amount, created_at).

id – первичный ключ,

user_id – UUID.

Загрузите 1000 тестовых строк (с дублями по user_id + created_at).

Напишите запрос, который удаляет дубликаты, оставляя строку с максимальным id.

```
with rowed_data as (
select id, user_id, amount, created_at,
rank() over(partition by user_id, created_at order by id desc) as row_num
from transactions
)

delete from transactions
where id in (select id from rowed_data where row_num > 1);
```

Напишите запрос, который добавляет колонку prev_amount (сумма предыдущей транзакции пользователя по дате created_at) и diff (разница).

```
select
id,
user_id,
amount,
created_at,
lag(amount) over (partition by user_id order by created_at) as prev_amount,
amount - lag(amount) over (partition by user_id order by created_at) as diff
from transactions;
```

Задача 2: Рекурсия

Создайте таблицу employees (id, name, manager_id). manager_id ссылка на id начальника.

Заполните иерархию (Гендиректор -> Директора -> Менеджеры).

Напишите рекурсивный запрос, который выводит всех подчиненных Гендиректора с уровнем вложенности (level).

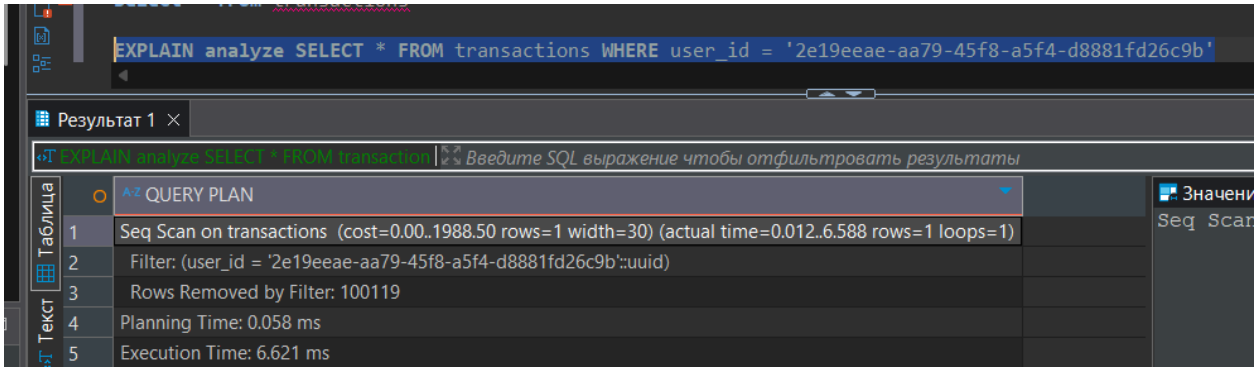
```
with recursive employees_tree_levels as (
select id, name, manager_id, 1 as level
from employees
where manager_id is null
union all
select e.id, e.name, e.manager_id, et.level + 1
```

```
from employees e
join employees_tree_levels et on e.manager_id = et.id
)
select * from employees_tree_levels order by level;
```

Задача 3: Оптимизация

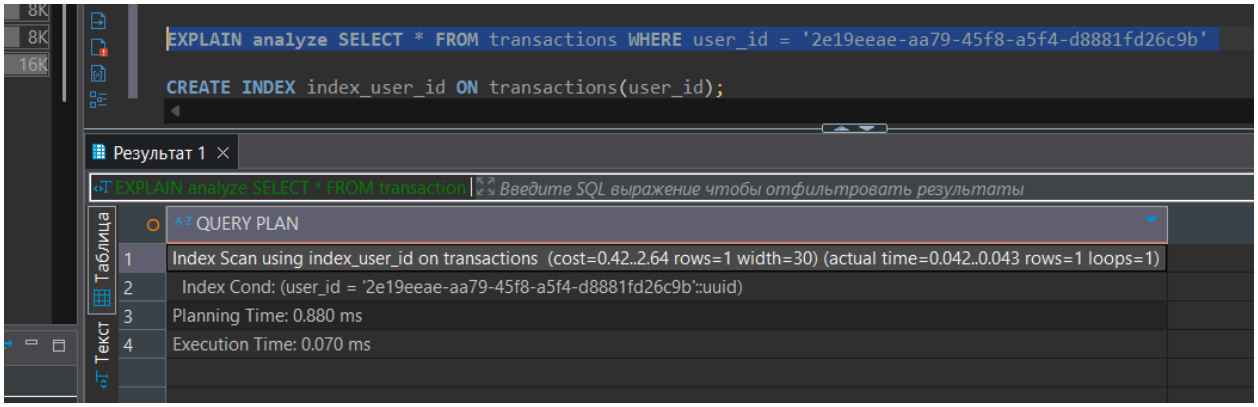
Добавьте в таблицу transactions ещё 100 тысяч строк.

Выполните SELECT * FROM transactions WHERE user_id = 'value' без индекса. Сделайте EXPLAIN ANALYZE. Зафиксируйте время и тип сканирования.



Создайте индекс по user_id.

Повторите запрос. Сравните план и время.



После добавления индекса плановое время выполнения увеличилось, но реальное время уменьшилось в десятки раз

Попробуйте «сломать» индекс, написав WHERE LOWER (user_id) = 'VALUE'. Объясните в комментарии, почему индекс не сработал.

```
EXPLAIN analyze SELECT * FROM transactions WHERE LOWER(user_id::text) = '2e19eeae-aa79-45f8-a5f4-d8881fd26c9b'

CREATE INDEX index_user_id ON transactions(user_id);
```

Результат 1

EXPLAIN analyze SELECT * FROM transactions | Введите SQL выражение чтобы отфильтровать результаты

	QUERY PLAN
1	Seq Scan on transactions (cost=0.00..2739.40 rows=501 width=30) (actual time=0.654..90.350 rows=1 loops=1)
2	Filter: (lower((user_id)::text) = '2e19eeae-aa79-45f8-a5f4-d8881fd26c9b'::text)
3	Rows Removed by Filter: 100119
4	Planning Time: 0.754 ms
5	Execution Time: 90.388 ms

Значение

Seq Scan

После добавления функции в условии это сломало индекс поиск опять совершается построково